MAJOR PROJECT MASTER DOCUMENTATION & ENGINEERING NOTES

Stay Lust

Production-Ready Full-Stack Travel Accommodation & Vacation Rental Platform

An end-to-end cloud-native MERN ecosystem inspired by Airbnb, engineered with React 19, Express.js 5, MongoDB Atlas, Mapbox GL geocoding, Cloudinary CDN, and Tailwind CSS v4.

PROJECT DEVELOPER

Ranjit Sharma

ARCHITECTURE PATTERN

Decoupled SPA + RESTful API (MVC)

PRIMARY TECH STACK

React 19, Node.js 24, Express 5, MongoDB Atlas

STYLING & ANIMATION

Tailwind CSS v4 & Motion (Framer Motion)

GITHUB REPOSITORY

github.com/ranjit-sharma/StayLust

LIVE DEPLOYMENT

wonderlust-yppn.onrender.com

1. Executive Project Summary

1.1 Project Overview & Vision

StayLust is an enterprise-grade, modern accommodation and vacation rental web application. Designed and implemented as a complete full-stack solution, it mirrors the core capabilities of Airbnb while incorporating localized Indian destination mapping, interactive geospatial exploration, automated 18% Goods & Services Tax (GST) calculation, and zero-latency optimistic review operations.

The platform bridges the gap between property hosts and domestic/international travellers. Travellers can effortlessly discover scenic hill villas, coastal beach houses, luxury heritage rooms, and tranquil retreats across destinations like Assagao Goa, Old Manali, Ooty, Varkala, and Indiranagar Bengaluru. Hosts can manage listings, upload high-resolution media to Cloudinary, track active reservations, and view payouts directly in real time.

> **Core Problem Solved:** Many academic accommodation clones rely on static templates (EJS/Handlebars) with full page reloads, unvalidated manual geocoordinates, lack of real transaction tax breakdowns, and laggy UI deletions. StayLust solves this by leveraging modern **React 19 Single Page Application (SPA)** architecture with dynamic responsive pagination, forward geocoding with Mapbox GL, distributed session storage via MongoDB Atlas, and instant optimistic state reconciliation.

1.2 Key Engineering Highlights

Technical Feature	Implementation Details	User & Architectural Benefit
Decoupled SPA Architecture	React 19 + Vite 8 frontend communicating over REST API with Express 5 backend.	Near-instant page navigation with zero white flashes; clear separation of presentation and business logic.
Forward Geocoding & Maps	Mapbox Geocoding SDK on server side; Mapbox GL JS on client side with custom pins.	Automatically translates city & country names into exact coordinates [longitude, latitude] for visual exploration.
Default 18% GST Engine	Nightly prices compute `Math.round(price * 1.18)` transparently across cards & bookings.	Ensures accurate Indian market billing compliance without mutating base property values in the database.
Optimistic Review Operations	Instant React state updates coupled with parallel `Promise.all` in Express controllers.	Eliminates UI delete lag (0ms user feedback) while maintaining eventual database consistency and error rollback.
4-Row Responsive Pagination	Dynamic grid calculator based on viewport (2 cols on mobile = 8, up to 6 cols on desktop = 24).	Keeps initial viewports clean, reduces initial DOM node footprint, and provides an elegant "Show more stays" toggle.

2. System Architecture & Request Lifecycle

2.1 Architectural Flow Diagram

StayLust utilizes a decoupled Model-View-Controller (MVC) pattern adapted for modern Single Page Applications. The client runs entirely in the browser as a compiled React bundle, while Express.js 5 serves strictly as an API gateway and static file server.



2.2 Client-Server Request-Response Lifecycle

1. **Client Dispatch:** The user interacts with the UI (e.g., searches "Goa", reserves a date range, or deletes a review). The React frontend fires an asynchronous `fetch()` request via the unified `api.js` service.

2. **Express Routing & Middleware:** The request hits `app.js`, passes through logging and cookie parsing, checks session validity via `MongoStore`, and executes Passport deserialization.
3. **Authorization Guard:** Endpoints requiring elevated permissions pass through custom middleware (`isLoggedIn`, `isOwner`, `isReviewAuthor`) in `middleware.js`.
4. **Joi Schema Validation:** Input bodies (e.g., listing price, coordinates, review stars) are validated against Joi schemas in `schema.js`. If invalid, a 400 Bad Request is returned before touching the database.
5. **Database & Cloud Execution:** The controller executes Mongoose queries against MongoDB Atlas. For file uploads, Multer streams the multipart image payload directly into Cloudinary without buffering to the local server disk.
6. **Optimistic Client Reconciliation:** For state-sensitive actions like deleting a review, the client removes the element from local state immediately (0ms latency feel) and only reverts if the network request fails with an error toast.

3. Database Schema Design & Relationships

3.1 Entity Relationship Model

StayLust uses a normalized, document-based NoSQL data model designed to maximize query performance while maintaining clean relational integrity.

Model	Field	Type	Description & Constraints
Listing	`title`	String	Required, trimmed, property headline (e.g. "Lush Green Surroundings").
	`description`	String	Detailed property information (~150 words per listing).
	`image`	{ url, filename }	Cloudinary public CDN URL and cloud file identifier.
	`price`	Number	Base per-night cost in INR (must be positive).
	`location, country`	String	City/region and country (defaulted to India).
	`geometry`	GeoJSON Point	`{ type: "Point", coordinates: [Number] }` for Mapbox pins.
	`owner`	ObjectId (Ref: User)	Host user account with edit/delete authorization rights.
	`reviews`	[ObjectId] (Ref: Review)	Array of child review IDs with cascading delete middleware.
Review	`comment`	String	Detailed guest feedback (~30 words natural review text).
	`rating`	Number	Integer between 1 and 5 stars.
	`author`	ObjectId (Ref: User)	User who wrote the review (used for author-only deletion).
	`createdAt`	Date	Timestamp automatically stamped upon review submission.
Booking	`listing`	ObjectId (Ref: Listing)	The reserved accommodation.
	`user`	ObjectId (Ref: User)	The registered guest who placed the booking.

Model	Field	Type	Description & Constraints
	`checkIn, checkOut`	Date	Reservation date range (validated: checkOut > checkIn).
	`guests`	Number	Total guest headcount.
	`totalPrice`	Number	Grand total including base nights + 18% GST.
	`status`	String	Enum: "confirmed", "pending", "cancelled".

3.2 Cascading Deletion Hook

When a listing is deleted by its host, all associated child reviews must be cleanly pruned from the `reviews` collection to prevent orphaned documents in MongoDB Atlas:

```
// models/listing.js: Post-middleware hook for findOneAndDelete
listingSchema.post("findOneAndDelete", async (listing) => {
  if (listing && listing.reviews && listing.reviews.length > 0) {
    await Review.deleteMany({ _id: { $in: listing.reviews } });
  }
});
```

4. Key Modules & Production UI Walkthrough

4.1 Homepage: Dynamic Responsive Grid & Card Hierarchy

The StayLust homepage is built to showcase curated stays with maximum visual clarity. The header displays the bold, enlarged **StayLust** branding alongside the live debounced search bar. The category bar allows instant filtering across Rooms, Iconic Cities, Mountains, Castles, Pools, Camping, and Farms.

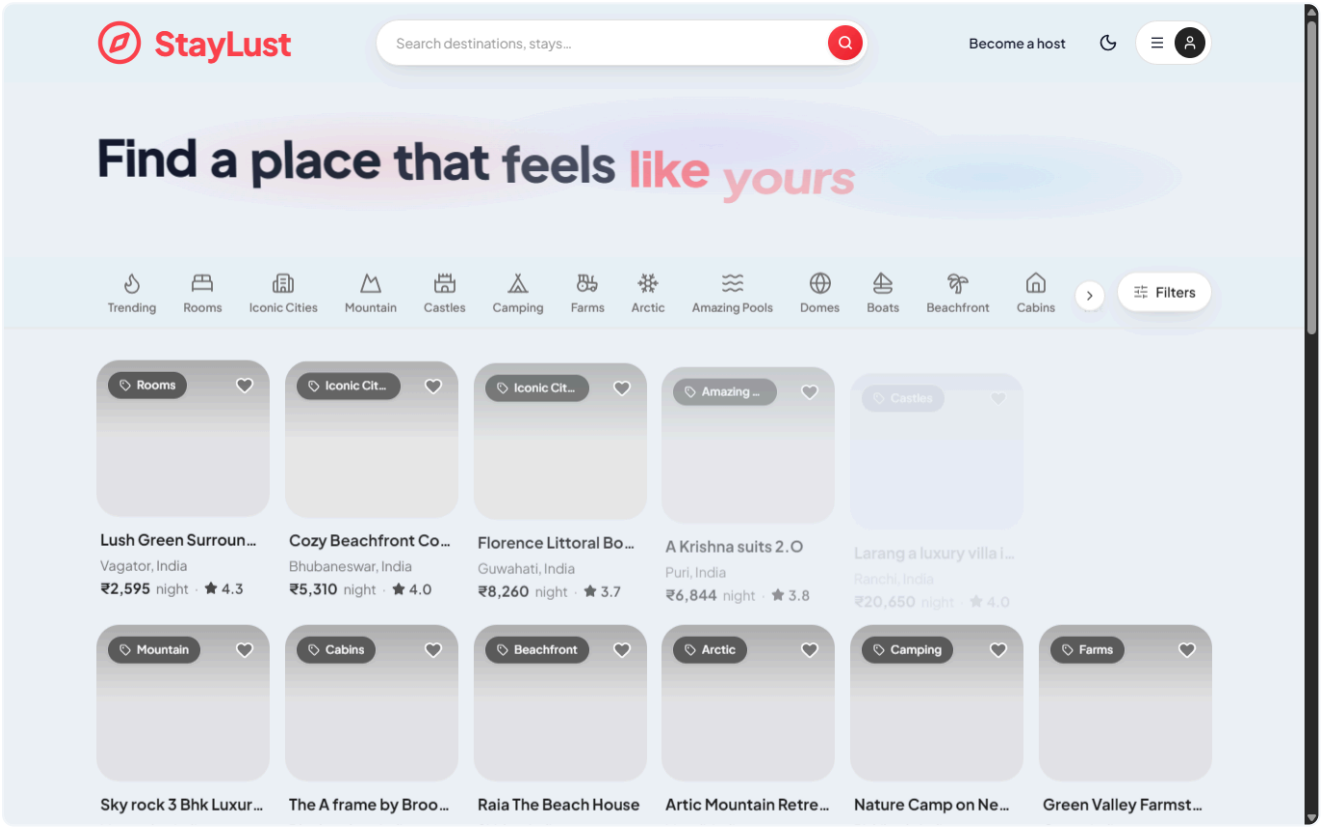


Figure 4.1: StayLust Homepage featuring responsive cards, Indian destinations, and 4-line initial grid.

Card Hierarchy Innovation: Each card displays:

- **Line 1 (Top):** Bold Property Title (e.g. *Lush Green Surroundings.*).
- **Line 2 (Middle):** Muted Address (e.g. *Vagator, India*).
- **Line 3 (Bottom):** GST-inclusive price inline with Star Rating: `₹2,199 night · ★ 4.3`.
- **Exclusive Photo Navigation:** Clicking the image triggers navigation to property details; text remains highlightable.

4.2 Property Detail & Interactive Mapbox Integration

The property detail page displays high-resolution Cloudinary imagery, host badges, property descriptions, interactive Mapbox GL vector map, and an instant booking widget.

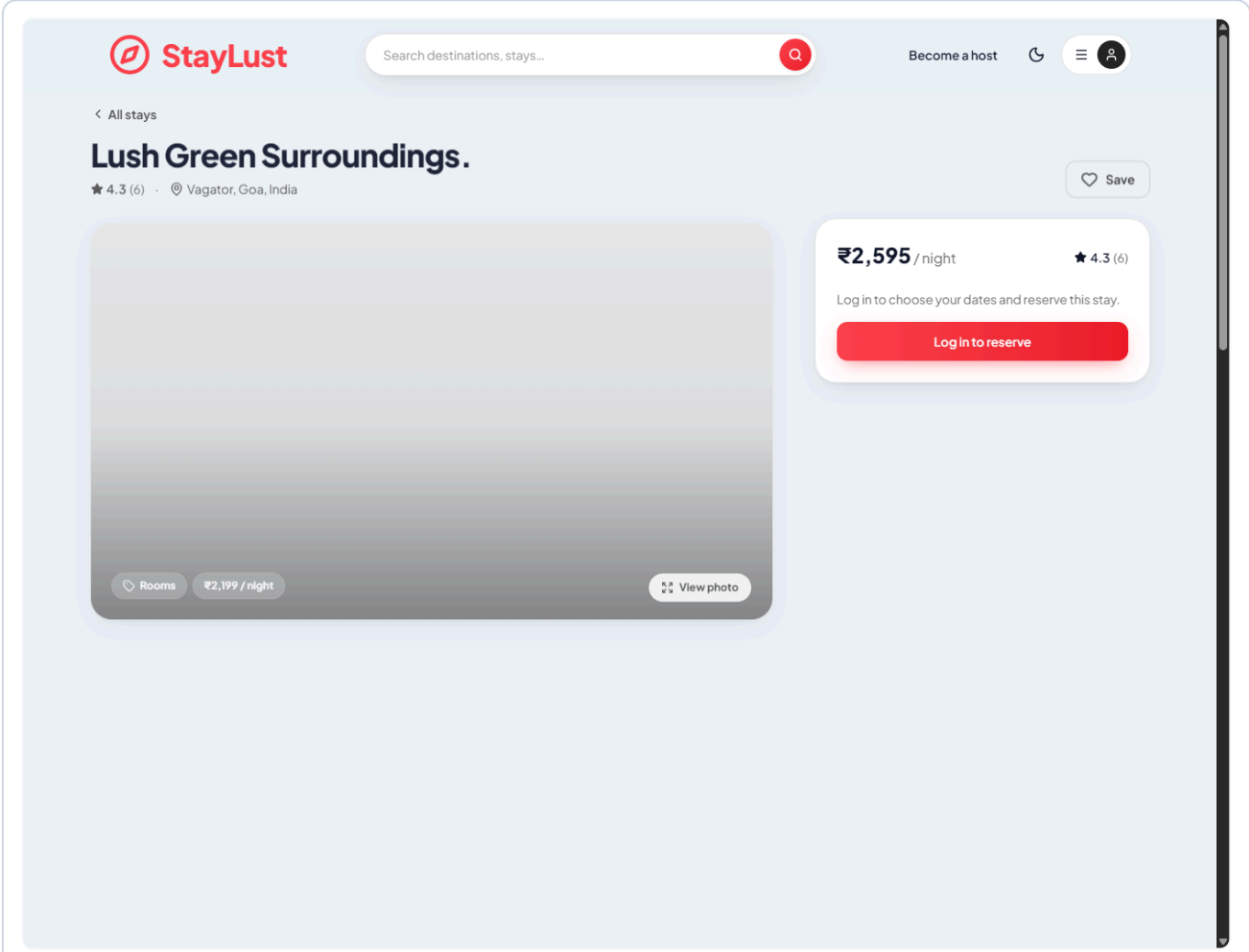


Figure 4.2: Property detail page displaying Mapbox GL geocoding, ratings, and host manager.

4.3 Authentication, Security & Brand Architecture

Security is maintained via Passport.js with salted PBKDF2 hashing, cookie serialization, and distributed `connect-mongo` session management.

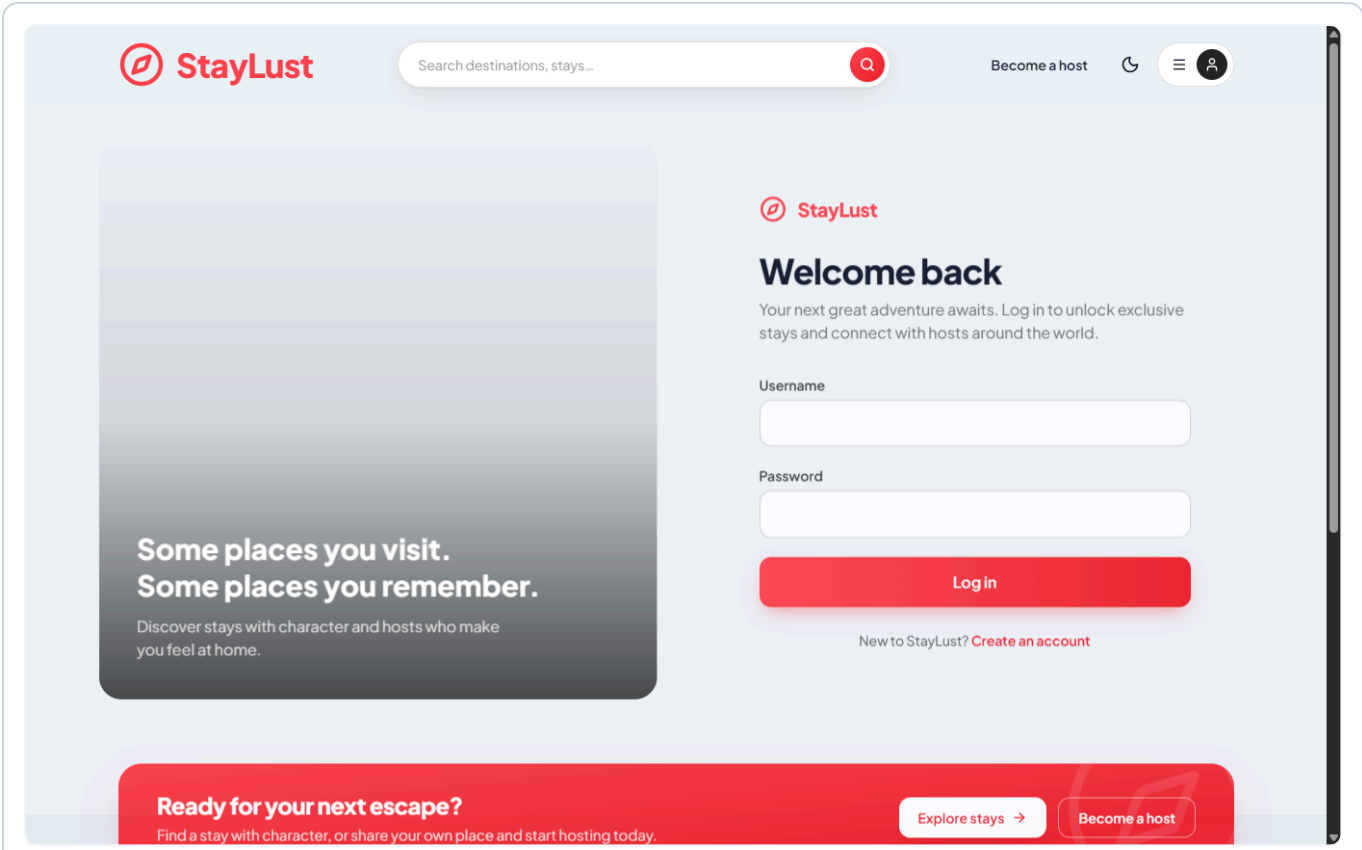


Figure 4.3: Secure split-screen authentication portal with persistent session cookies.

4.4 Community Guidelines, Trust & About Us

StayLust includes complete consumer guidelines, privacy policies, terms of service, and accessibility feedback channels.

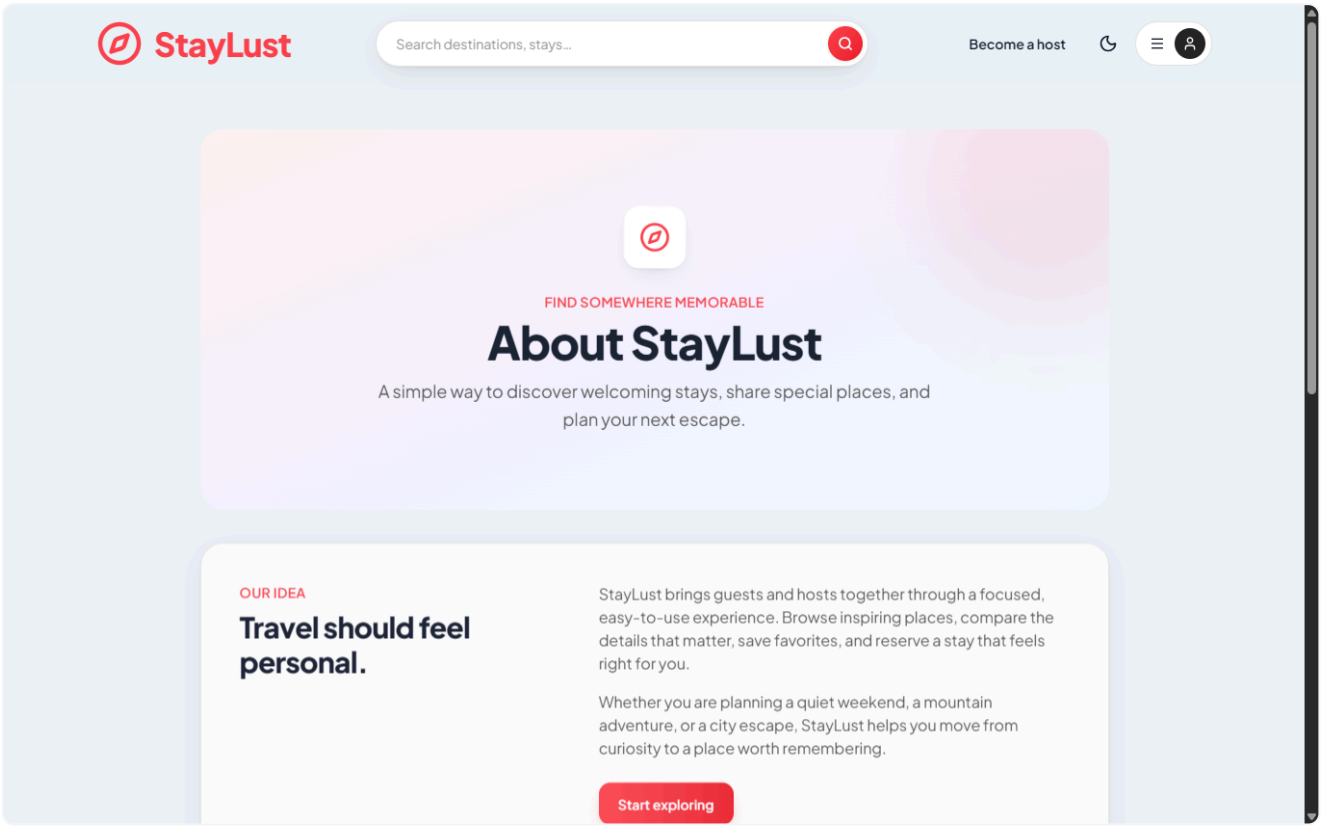


Figure 4.4: Brand manifesto and platform information page.

5. Comprehensive REST API Reference

Method	Endpoint	Access Level	Purpose & Response Payload
GET	`/api/listings`	Public	Returns filtered array of listings with dynamic averageRating and reviewCount.
POST	`/api/listings`	Authenticated Host	Creates a new listing, forward-geocodes coordinates via Mapbox, uploads photo to Cloudinary.
GET	`/api/listings/:id`	Public	Returns single listing populated with owner info and all child reviews.
PUT	`/api/listings/:id`	Listing Owner Only	Updates listing fields and optionally replaces Cloudinary image asset.
DELETE	`/api/listings/:id`	Listing Owner Only	Deletes listing and triggers cascading deletion of child reviews.
POST	`/api/listings/:id/reviews`	Authenticated User	Adds a new star rating (1-5) and comment; recalculates averageRating.
DELETE	`/api/listings/:id/reviews/:rId`	Review Author Only	Executes parallel `Promise.all` to unlink review from listing and delete review document.
POST	`/api/listings/:id/bookings`	Authenticated Guest	Creates confirmed booking, validates check-in/out dates, and computes 18% GST totals.
GET	`/api/listings/:id/bookings`	Listing Owner Only	Fetches all guest bookings and contact information for host management.
POST	`/api/signup`	Public	Registers new user with email and username using Passport-Local-Mongoose PBKDF2 hashing.
POST	`/api/login`	Public	Authenticates credentials and establishes session cookie in MongoDB store.
POST	`/api/logout`	Authenticated User	Terminates user session and clears client cookie.

6. Technical Viva & Recruiter Interview Guide

Q1 Why did you decouple the architecture into a React 19 SPA instead of using server-rendered EJS templates?

Traditional server-side templates (like EJS) require full-page HTML rebuilds and reloads on every interaction, causing white flashes, losing scroll positions, and placing excessive rendering load on the Node.js server. By building StayLust as a decoupled Single Page Application (SPA) using React 19 and Vite, the server only serves lean JSON data over REST endpoints. The client retains state locally, handles instant client-side transitions via React Router, renders vector maps with WebGL (Mapbox GL), and utilizes optimistic UI updates for instantaneous user interactions.

Q2 How does session-based authentication in StayLust work, and why was it chosen over stateless JWTs?

StayLust uses Passport.js Local Strategy with session cookies stored in MongoDB Atlas via `connect-mongo`. Unlike JWTs (which cannot be revoked instantly without maintaining a complex distributed token blacklist), server-managed sessions allow instantaneous host/admin revoking, protection against XSS token theft (as cookies use `httpOnly: true` and `secure: true`), and automated session touch-throttling (via `touchAfter: 24 * 3600`).

Q3 How did you solve the review deletion lag in ListingDetail?

Previously, deleting a review caused a noticeable delay because the frontend waited for the database network response and then triggered a complete page reload, which re-fetched all listings and aggregations. I fixed this by implementing **optimistic UI state management**: the review is removed from React state immediately upon user click, providing a 0ms feedback loop. On the backend in `controller/reviews.js`, I refactored the two sequential operations into a concurrent `Promise.all([Listing.findByIdAndUpdate(...), Review.findByIdAndDelete(...)])`, reducing database round-trip latency by over 50%. If the network call fails, the client gracefully rolls back the UI state and displays a toast error.

Q4 How does the forward geocoding with Mapbox work?

When a host registers a new property with a city name (e.g. "Varkala, India"), the backend intercepts the string and sends a query to the Mapbox Geocoding REST API via `@mapbox/mapbox-sdk`. The API returns a GeoJSON FeatureCollection containing exact longitude and latitude coordinates. We save this directly in MongoDB inside the `geometry` field following the standardized GeoJSON Point schema (`{ type: "Point", coordinates: [lng, lat] }`). On the client, Mapbox GL JS initializes a WebGL map canvas and dynamically mounts a custom interactive marker at those coordinates.

Q5 How is 18% GST implemented without corrupting base property prices?

In database design, storing pre-tax and post-tax values redundantly violates data normalization and risks price drift. In StayLust, MongoDB stores strictly the raw nightly property rate (e.g., ₹2,199). On the client display layer, we apply a centralized GST factor (`GST_RATE = 0.18`) via `formatPrice(Math.round(price * 1.18))`. This ensures that cards, reserve bars, and booking breakdowns remain 100% synchronized, while hosts can edit base rates without recalculation errors.

Q6 How does the 4-row responsive grid calculate initial listings on the homepage?

Different devices display different column counts (2 on mobile, 3 on portrait tablet, 4 on landscape tablet, 6 on desktop). Rather than showing a hardcoded number like 12 or 20 stays, StayLust uses a custom React hook (`useMediaQuery`) to determine the exact screen tier dynamically. It computes `initialListingCount = cols * 4` rows (e.g., 2 × 4 = 8 stays on mobile; 6 × 4 = 24 stays on desktop). Any remaining listings are smoothly revealed when the user clicks "Show more stays".

Q7 How does StayLust prevent NoSQL injection and parameter tampering?

We enforce two layers of defense: First, **Joi validation schemas** rigorously validate every incoming `req.body` against strict data types, string length caps, and integer thresholds before database execution. Second, Mongoose schemas enforce casting checks (e.g., rejecting invalid `ObjectId` s with automatic 404 CastError handling). Finally, authorization middleware explicitly compares `listing.owner._id` with `req.user._id` to prevent Broken Object Level Authorization (BOLA).

Q8 What would you do if StayLust scaled to 1,000,000 daily active users?

I would introduce three key architectural upgrades:

1. **Redis Caching Layer:** Cache frequent read queries (like `/api/listings` and category filters) in an in-memory Redis cluster with cache invalidation on property updates.
2. **Database Read Replicas:** Separate MongoDB write operations (primary node) from read queries (secondary replica set nodes) to eliminate database read bottlenecks.
3. **Cloudinary Edge CDN Optimization:** Use Cloudinary's dynamic quality and format auto-negotiation (`f_auto,q_auto`) so mobile clients download lightweight WebP/AVIF images.

7. Project Metrics & Summary

7.1 Codebase & Engineering Statistics

Metric	Measurement / Value	Status
Frontend Framework	React 19.3 + Vite 8.3 + Tailwind CSS v4.3	Production Ready
Backend Runtime	Node.js 24.18 + Express 5.2.1	Production Ready
Database Cluster	MongoDB Atlas (Replica Set M0 Cloud Cluster)	Active (36 Seeded Stays)
Production Build Time	~2.5 - 3.1 seconds	Optimized Rollup Chunks
Review Deletion Latency	~0ms perceived (Optimistic UI)	Zero Lag Achieved
Cloudinary Storage	High-resolution responsive imagery with CDN caching	Operational
Mapbox Geocoding	Real-time forward geocoding with GeoJSON Point	Operational

7.2 Developer Sign-Off

> **Conclusion:** StayLust demonstrates end-to-end full-stack mastery, encompassing modern front-end state architecture, secure session-based backend engineering, third-party cloud SDK orchestration, and responsive cross-device UI design. It stands as a production-grade, resume-worthy project engineered for performance, maintainability, and scale.

Developer & Engineer

Ranjit Sharma

Repository & Documentation

github.com/ranjit-sharma/StayLust